MODUL 11

TAHAP 2 – MENGENAL BASIS DATA BERORIENTASI OBJEK

1. Pengertian Basis Data Berorientasi Objek

Basis data berorientasi objek (Object-Oriented Database) adalah pendekatan penyimpanan data yang merepresentasikan informasi dalam bentuk objek. Objek memiliki beberapa karakteristik:

- **Identity:** identitas unik suatu objek.
- **Attribute:** data atau properti yang dimiliki objek.
- **Method:** operasi atau perilaku yang dapat dilakukan objek.
- **Encapsulation:** penggabungan data dan perilaku dalam satu objek.
- **Inheritance:** pewarisan karakteristik dari objek atau kelas induk.
- **Polymorphism:** kemampuan objek memberikan implementasi perilaku yang berbeda.

Contoh pada Green Map PNM: objek Tanaman memiliki atribut dasar dan perilaku, lalu objek turunan menambahkan atribut sesuai jenisnya.

Tanaman

---

id_tanaman

nama_tanaman

jenis_tanaman

kondisi

latitude

longitude

---

tampilkanInformasi()

laporkanKondisi()

Objek turunannya:

Pohon	TanamanHias	TanamanObat
(atribut Tanaman)	(atribut Tanaman)	(atribut Tanaman)
tinggi_m	warna_bunga	bagian_dipakai
diameter_cm	musim_berbunga	khasiat

2. Perbedaan Basis Data Relasional dan Berorientasi Objek

Aspek	Relasional	Berorientasi objek	Contoh pada Green Map PNM
Struktur data	Tabel	Objek	Tabel tanaman, objek Tanaman
Identitas	Primary Key	Object identity	id_tanaman
Hubungan	Foreign Key	Object reference	perawatan.id_tanaman merujuk ke tanaman
Atribut	Kolom	Properti objek	nama_tanaman, kondisi
Perilaku	Umumnya di aplikasi atau stored program	Dapat menjadi bagian model objek	fn_jumlah_laporan() sebagai stored function
Pewarisan	Bukan fitur utama model relasional	Mendukung inheritance	Pohon dan TanamanHias turunan Tanaman
Contoh DBMS	MySQL, PostgreSQL	ObjectDB, db4o (historis)	Green Map PNM memakai MySQL

3. Memahami Pendekatan Penyimpanan Objek pada DBMS Relasional

MySQL/MariaDB merupakan DBMS relasional, bukan Object-Oriented DBMS murni. Namun, DBMS relasional dapat dipakai untuk menyimpan representasi objek menggunakan tipe data JSON. Pada Green Map PNM, atribut dasar tanaman disimpan sebagai kolom biasa, sedangkan atribut yang berbeda tiap jenis disimpan sebagai JSON.

```
{
  "id_tanaman": 1,
  "nama_tanaman": "Trembesi",
  "jenis_tanaman": "Pohon",
  "atribut": {
    "tinggi_m": 12,
    "diameter_cm": 45
  }
}
```

Dengan JSON, atribut suatu tanaman dapat memiliki struktur yang berbeda tanpa harus membuat banyak kolom kosong pada satu tabel. Pohon tidak perlu kolom warna_bunga, dan tanaman hias tidak perlu kolom diameter_cm.

4. Mengenal JSON dalam MySQL/MariaDB

JSON (JavaScript Object Notation) adalah format pertukaran data yang menggunakan pasangan key-value dan struktur array. Contoh data tanaman obat:

```
{
  "nama": "Jahe",
  "obat": true,
  "bagian": {
    "dipakai": "rimpang",
    "warna": "kuning"
  },
  "khasiat": [
    "anti-mual",
    "hangat"
  ]
}
```

Fungsi JSON yang dapat digunakan:

Fungsi	Kegunaan
JSON_OBJECT()	Membuat objek JSON.
JSON_ARRAY()	Membuat array JSON.
JSON_VALID()	Memeriksa validitas JSON.
JSON_EXTRACT()	Mengambil nilai dari JSON.
JSON_SET()	Menambahkan atau mengubah nilai JSON.
JSON_REMOVE()	Menghapus elemen JSON.
JSON_CONTAINS()	Memeriksa keberadaan nilai dalam JSON.
JSON_LENGTH()	Menghitung jumlah elemen JSON.

Perlu diperhatikan bahwa implementasi tipe JSON MySQL dan MariaDB tidak sepenuhnya identik. MySQL menyediakan tipe JSON native, sedangkan MariaDB menggunakan representasi JSON berbasis teks dengan validasi JSON pada implementasi yang didukung versinya.

5. Praktik JSON pada Data Green Map PNM

Praktik dilakukan pada tabel percobaan tanaman_objek agar tabel utama db_green_map_pnm tidak berubah.

5.1 Membuat tabel dan mengisi data (JSON_OBJECT dan JSON_ARRAY)

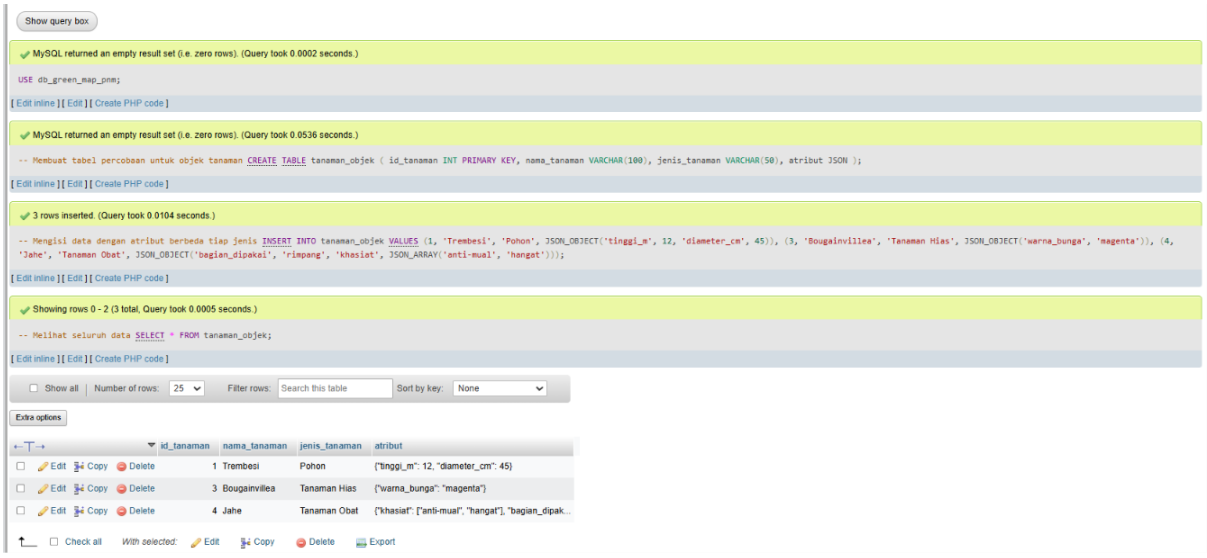
USE db_green_map_pnm;

```
-- Membuat tabel percobaan untuk objek tanaman
CREATE TABLE tanaman_objek (
  id_tanaman INT PRIMARY KEY,
  nama_tanaman VARCHAR(100),
  jenis_tanaman VARCHAR(50),
  atribut JSON
);

-- Mengisi data dengan atribut berbeda tiap jenis
INSERT INTO tanaman_objek VALUES
(1, 'Trembesi', 'Pohon', JSON_OBJECT('tinggi_m', 12, 'diameter_cm', 45)),
(3, 'Bougainvillea', 'Tanaman Hias', JSON_OBJECT('warna_bunga', 'magenta')),
(4, 'Jahe', 'Tanaman Obat', JSON_OBJECT('bagian_dipakai', 'rimpang',
                                        'khasiat', JSON_ARRAY('anti-mual', 'hangat')));

-- Melihat seluruh data
SELECT * FROM tanaman_objek;
```

BUKTI SCREENSHOT

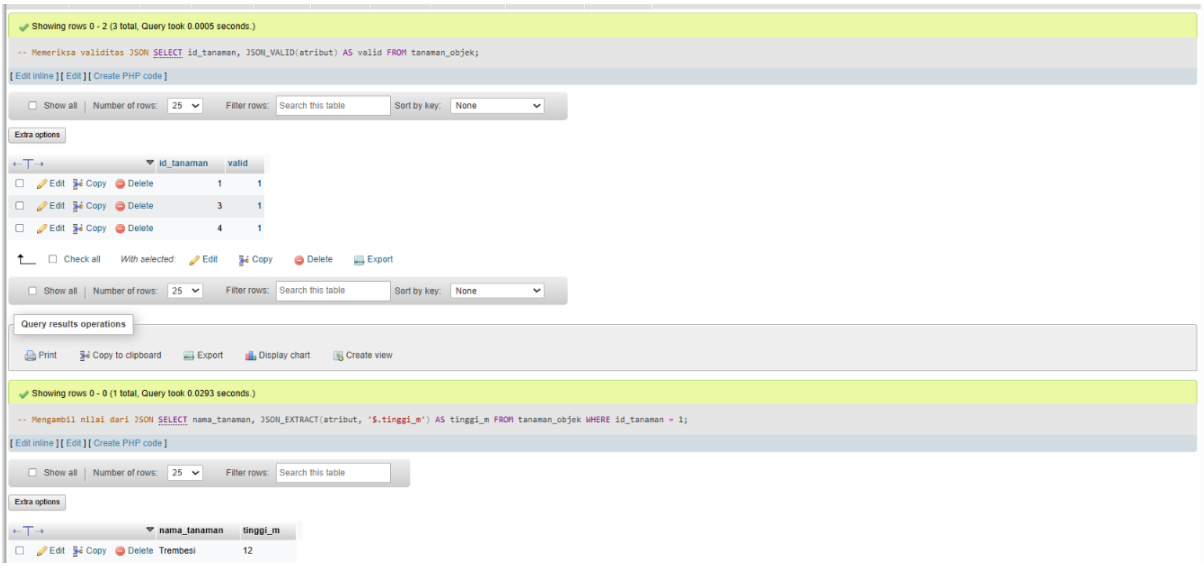


5.2 Memeriksa dan mengambil nilai (JSON_VALID dan JSON_EXTRACT)

```
-- Memeriksa validitas JSON
SELECT id_tanaman, JSON_VALID(atribut) AS valid FROM tanaman_objek;

-- Mengambil nilai dari JSON
SELECT nama_tanaman, JSON_EXTRACT(atribut, '$.tinggi_m') AS tinggi_m
FROM tanaman_objek WHERE id_tanaman = 1;
```

BUKTI SCREENSHOT

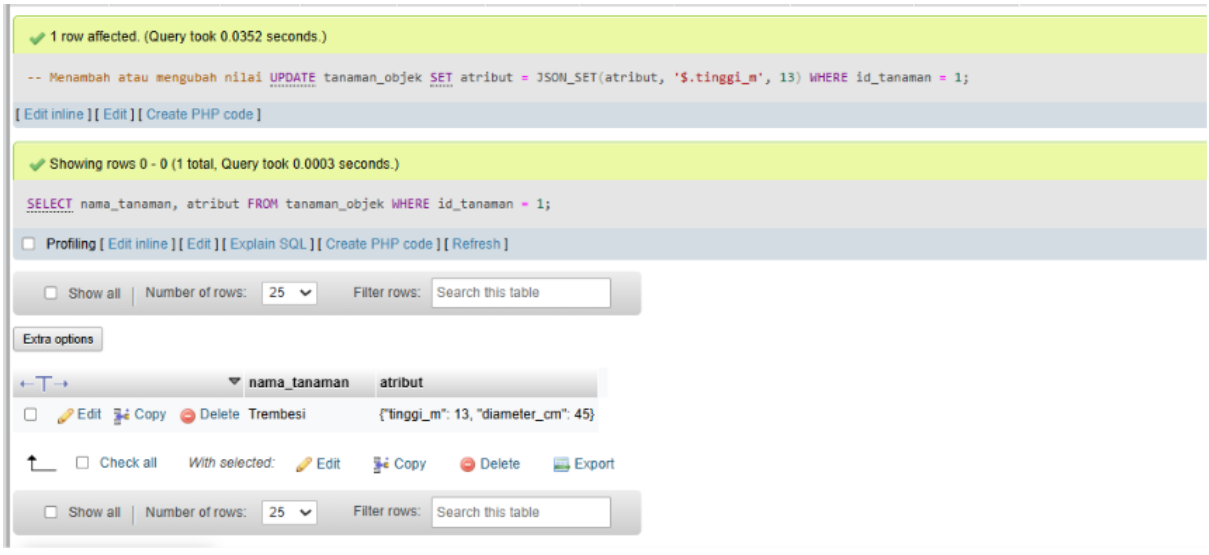


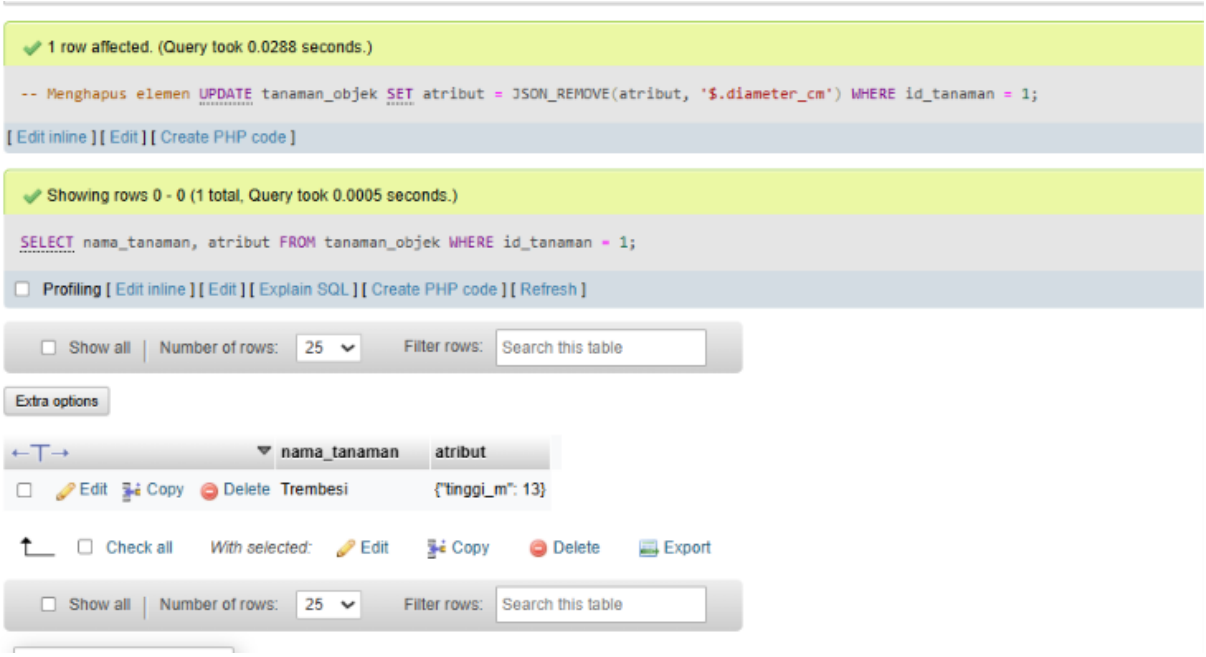
5.3 Mengubah dan menghapus nilai (JSON_SET dan JSON_REMOVE)

-- Menambah atau mengubah nilai
UPDATE tanaman_objek SET atribut = JSON_SET(atribut, '$.tinggi_m', 13) WHERE id_tanaman = 1;
SELECT nama_tanaman, atribut FROM tanaman_objek WHERE id_tanaman = 1;

-- Menghapus elemen
UPDATE tanaman_objek SET atribut = JSON_REMOVE(atribut, '$.diameter_cm') WHERE id_tanaman = 1;
SELECT nama_tanaman, atribut FROM tanaman_objek WHERE id_tanaman = 1;

BUKTI SCREENSHOT



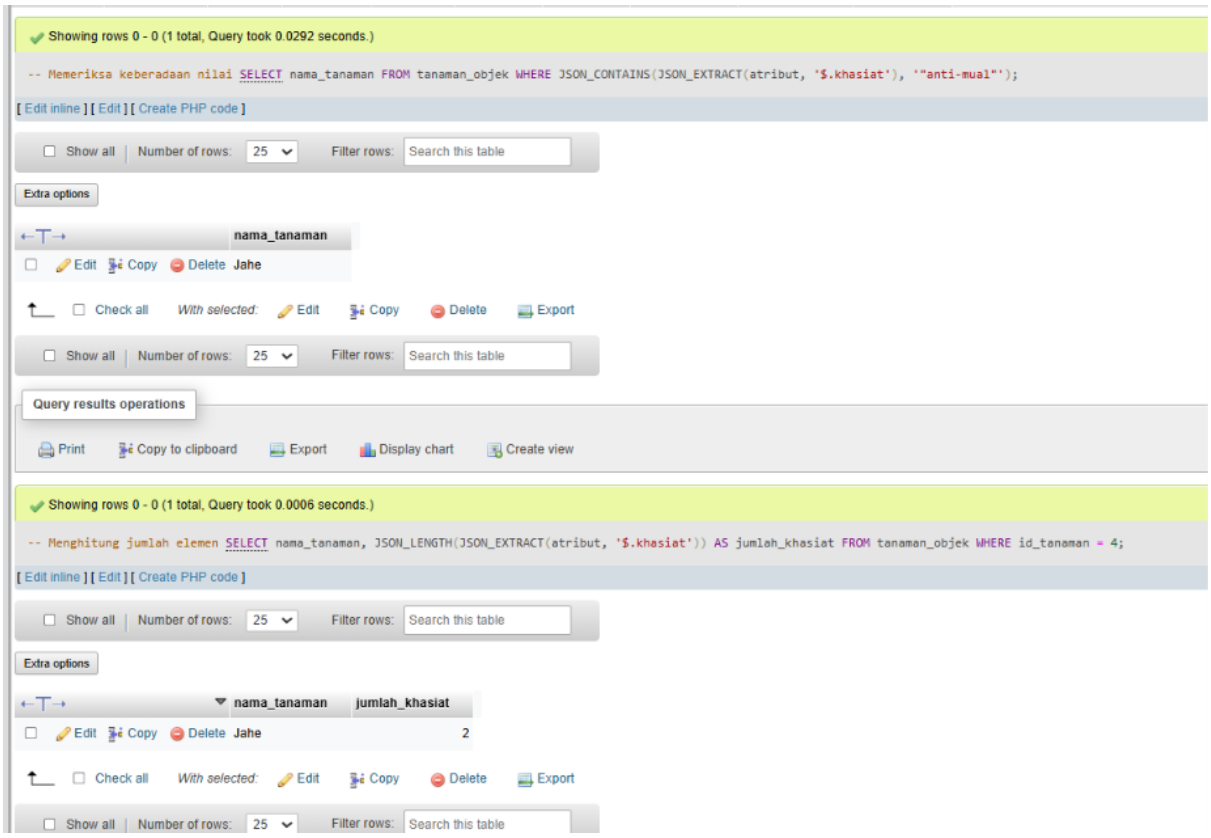


5.4 Memeriksa dan menghitung elemen (JSON_CONTAINS dan JSON_LENGTH)

-- Memeriksa keberadaan nilai
SELECT nama_tanaman FROM tanaman_objek
WHERE JSON_CONTAINS(JSON_EXTRACT(atribut, '$.khasiat'), '"anti-mual"');

-- Menghitung jumlah elemen
SELECT nama_tanaman, JSON_LENGTH(JSON_EXTRACT(atribut, '$.khasiat')) AS jumlah_khasiat
FROM tanaman_objek WHERE id_tanaman = 4;

BUKTI SCREENSHOT



6. Kesimpulan

Basis data berorientasi objek menyimpan data sebagai objek yang memiliki identitas, atribut, perilaku, dan pewarisan, sedangkan basis data relasional menyimpan data dalam tabel yang dihubungkan Primary Key dan Foreign Key. MySQL/MariaDB adalah DBMS relasional, tetapi dapat menyimpan representasi objek melalui tipe JSON. Pada Green Map PNM, pendekatan ini berguna untuk atribut tanaman yang berbeda tiap jenis (pohon, tanaman hias, tanaman obat) tanpa menambah banyak kolom kosong, sementara data inti tetap disimpan pada kolom relasional agar pencarian dan integritasnya terjaga. Pendekatan ini tetap bersifat semi-terstruktur dan tidak mengubah MySQL menjadi basis data berorientasi objek.